

Calidad de Software

Ingeniería de Software 4

Luz Mery Rios Alzate / Ingeniería de Sistemas

Estudio y Exposición de un Método de Gestión de Riesgos

Investigar, comprender y explicar un método de gestión de riesgos en proyectos de software, desarrollando la capacidad de identificar, analizar, evaluar, responder y hacer seguimiento a los riesgos que pueden afectar el proyecto, mediante una exposición práctica y didáctica.

Métodos que pueden ser seleccionados:

- ✓ Matriz de Probabilidad–Impacto.
- ✓ Análisis cualitativo de riesgos.
- ✓ Análisis cuantitativo de riesgos.
- ✓ Método RMMM — Risk Mitigation, Monitoring and Management.
- ✓ Árbol de decisión.
- ✓ Monte Carlo.
- ✓ FMEA / AMFE.
- ✓ Registro de riesgos (Risk Register).
- ✓ Otra técnica de gestión de riesgos previamente aprobada.



Propósito de la actividad: No se trata solamente de explicar la técnica, sino de enseñarla a los compañeros y demostrar cómo se utiliza en un proyecto de software mediante un ejemplo práctico.



2

Estudio y Exposición de un Método de Gestión de Riesgos

Cada estudiante deberá preparar un mini-taller para sus compañeros, que incluya:

1

A. INTRODUCCIÓN

- ¿Qué es la gestión de riesgos?
- ¿Qué es un riesgo en un proyecto de software?
- ¿Para qué sirve el método?
- ¿En qué momento del proyecto puede utilizarse?
- ¿Qué tipo de riesgos permite analizar?
- ¿Qué información requiere?



2

B. FUNDAMENTOS

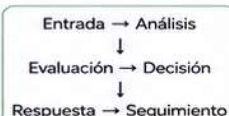
- Explicar los conceptos fundamentales relacionados con el método:
- Riesgo.
 - Probabilidad.
 - Impacto.
 - Exposición al riesgo.
 - Causas y consecuencias.
 - Nivel de riesgo.
 - Tratamiento o respuesta al riesgo.



3

C. PROCEDIMIENTO

- Explicar paso a paso cómo aplicar el método. Se recomienda utilizar un esquema, diagrama o flujo de proceso.



4

D. EJEMPLO COMPLETO

- Desarrollar un caso de un proyecto de software que incluya:
- Contexto del proyecto.
 - Identificación de los riesgos.
 - Datos iniciales.
 - Aplicación del método.
 - Cálculos o clasificación.
 - Priorización de los riesgos.
 - Decisiones tomadas.
 - Estrategia de respuesta.
 - Resultado final.



5

E. TALLER PARA LOS COMPAÑEROS

- Preparar un pequeño ejercicio práctico para que los compañeros apliquen el método durante la exposición. El ejercicio debe permitir que los compañeros:
- 1 Identifiquen uno o varios riesgos.
 - 2 Apliquen el método.
 - 3 Analicen los resultados.
 - 4 Tomen una decisión.
 - 5 Propongan una respuesta al riesgo.



6

F. VENTAJAS Y LIMITACIONES

- Ventajas.
- Desventajas.
- Limitaciones.
- ¿Cuándo es recomendable utilizarlo?
- ¿Cuándo no sería la mejor alternativa?
- ¿Qué tipo de proyecto de software se beneficia de su utilización?



7

G. DECISIÓN Y RESPUESTA AL RIESGO (TRANSVERSAL)

- Explicar qué se hará con cada riesgo identificado.
- Evitar
 - Mitigar
 - Transferir
 - Aceptar



Justificar por qué se seleccionó esa respuesta.



Incluir en la exposición:

2 LIBROS

relacionados con gestión de riesgos, ingeniería de software o gestión de proyectos.

2 ARTÍCULOS CIENTÍFICOS O TRABAJOS DE GRADO relacionados directamente con el método seleccionado.



Importante: las fuentes no deben limitarse a mencionarse en la bibliografía; durante la exposición deben explicar qué aporte hace cada fuente al método estudiado.

Verificación y Validación



1. no implica la ejecución de código
2. Utiliza métodos como revisiones, recorridos, inspecciones y verificación documental,
3. verifica si el software confirma una especificación
4. encuentra los errores al principio del ciclo de desarrollo.
5. la realiza el equipo de control de calidad.

1. implica la ejecución de código.
2. Utiliza métodos como caja negra y caja blanca y pruebas no funcionales
3. verifica si el software cumple con los requisitos y expectativas.
4. encuentra los errores que la verificación no puede detectar.
5. se realiza mediante la participación del equipo de pruebas con el equipo de control de calidad.

A la Verdad, por la fe y la ciencia



Técnica de Verificación

Revisiones de Software



Ayudan a encontrar falencia en el software.



Sirven para verificar la calidad y/o el estado del producto.



Revisiones de Software: Software
Technical Reviews. IEEE Standard 1028-
2008 for Software Reviews and Audits.

Revisiones entre pares
Revisión Administrativa
Inspecciones



Técnicas de Validación

Estudio y Exposición de la Norma ISO/IEC/IEEE 29119 (Pruebas de Software)

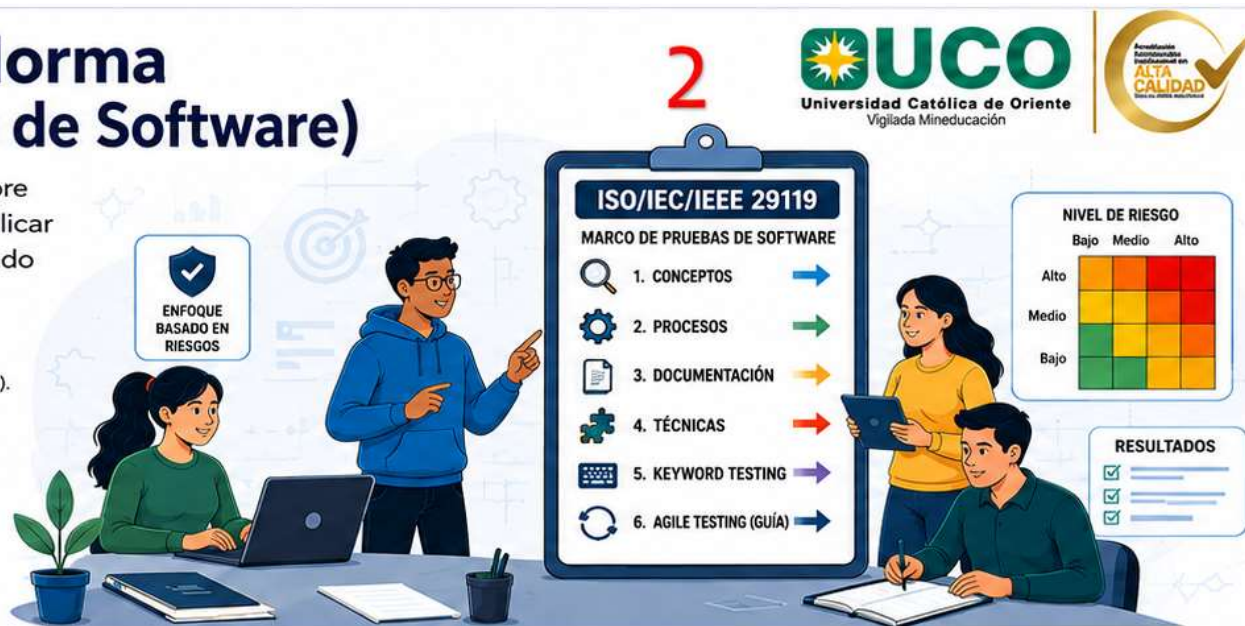
Investigar, comprender y explicar la norma ISO/IEC/IEEE 29119 sobre Pruebas de Software, desarrollando la capacidad de entender y aplicar sus conceptos, procesos, técnicas, documentación y enfoque basado en riesgos, mediante una exposición práctica y didáctica.

Aspectos de la norma que pueden ser seleccionados:

- ✓ Conceptos generales (Parte 1).
- ✓ Procesos de prueba (Parte 2).
- ✓ Documentación de pruebas (Parte 3).
- ✓ Técnicas de prueba (Parte 4).
- ✓ Keyword-Driven Testing (Parte 5).
- ✓ Pruebas en contexto ágil (Parte 6 - Guía).
- ✓ Enfoque basado en riesgos.
- ✓ Roles y responsabilidades.
- ✓ Niveles de prueba.
- ✓ Gestión de defectos.



Propósito de la actividad: No se trata solamente de explicar la norma, sino de **enseñarla** a los compañeros y demostrar cómo se utiliza en un proyecto de software mediante un ejemplo práctico.



Estudio y Exposición de la Norma ISO/IEC/IEEE 29119 (Pruebas de Software)

Cada estudiante deberá preparar un mini-taller para sus compañeros, que incluya:

1

A. INTRODUCCIÓN

- ¿Qué es la norma ISO/IEC/IEEE 29119?
- ¿Cuál es su propósito?
- ¿Por qué es importante en proyectos de software?
- ¿Qué problemas ayuda a resolver?
- ¿En qué contextos puede aplicarse?



2

B. CONCEPTOS GENERALES (Parte 1)

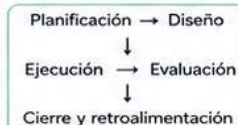
- Vocabulario y definiciones clave.
- Principios y fundamentos.
- Enfoque basado en riesgos.
- Características de los sistemas y del contexto.
- Niveles y tipos de prueba.



3

C. PROCESOS DE PRUEBA (Parte 2)

Explicar el ciclo y los procesos definidos en la norma:



4

D. DOCUMENTACIÓN DE PRUEBAS (Parte 3)

Presentar los documentos sugeridos y sus plantillas:

- Plan de prueba.
- Diseño de prueba.
- Casos de prueba.
- Procedimientos.
- Registro de ejecución.
- Reporte de incidentes.
- Reporte de prueba.



5

E. TÉCNICAS DE PRUEBA (Parte 4)

Explicar y ejemplificar técnicas:

- 1 División de equivalencias.
- 2 Análisis de valores límite.
- 3 Tablas de decisión.
- 4 Transición de estados.
- 5 Cobertura estructural.
- 6 Basadas en experiencia.



6

F. KEYWORD TESTING (Parte 5) y ÁGIL (Parte 6)

- ¿Qué es el Keyword-Driven Testing?
- Beneficios y componentes.
- Ejemplo de tabla de palabras clave.
- ¿Cómo aplicar la norma en proyectos ágiles?



7

G. DECISIÓN Y RESPUESTA AL RIESGO (TRANSVERSAL)

Proponer cómo la norma ayuda a cada riesgo identificado:

- Evitar
- Mitigar
- Transferir
- Aceptar



Justificar por qué se seleccionó esa respuesta.



Incluir en la exposición:

2 LIBROS

relacionados con pruebas de software, calidad de software o gestión de pruebas.

2 ARTÍCULOS CIENTÍFICOS O TRABAJOS DE GRADO relacionados directamente con la norma 29119.



Importante: las fuentes no deben limitarse a mencionarse en la bibliografía; durante la exposición deben explicar qué aporte hace cada fuente al tema estudiado.

Pruebas de Software

OBJETIVO



Asegurar la calidad del software verificando y validando que cumple con los requisitos especificados.

IMPORTANCIA



Identificación temprana de defectos, reducción de costos de mantenimiento, y aumento de la satisfacción del cliente.

A la Verdad, por la fe y la ciencia

Pruebas de Software



Ariane 5.

Lanzado por primera vez el 4 de junio de 1996.

Ariane 5.

se autodestruyó 36.7 segundos.



Motivo:

Un error en el software de control del cohete debido a un fallo en la conversión de datos. El software intentó convertir un número flotante de 64 bits a un entero de 16 bits sin comprobar si el número era demasiado grande para el nuevo formato, provocando una excepción que no fue manejada correctamente.

Consecuencia:

Pérdida total del cohete y su carga, valorada en aproximadamente 500 millones de dólares.

Verdad por la fe y la ciencia

Pruebas de Software

El Caso del USS Yorktown (1997)

El sistema de control del USS Yorktown, un crucero de la Marina de los EE.UU., se paralizó debido a un error de división por cero en el software de su sistema de control.

Motivo:

Una entrada incorrecta de un operador humano que introdujo un valor de cero en un campo de entrada, lo que provocó un error de división por cero no manejado.

Consecuencia:

El buque quedó inmovilizado en el océano durante varias horas, lo que afectó gravemente las operaciones militares y la imagen de la Marina.



Pruebas de Software



El Error del Radar Patriot (1991)

Durante la Guerra del Golfo, un misil iraquí Scud impactó en un cuartel del ejército estadounidense en Dhahran, Arabia Saudita, causando la muerte de 28 soldados.

Motivo:

Un error de precisión en el software del sistema de radar del misil Patriot que aumentaba con el tiempo. Después de estar operativo por más de 100 horas, el error acumulado hacía que el sistema no pudiera detectar correctamente el misil entrante.

Consecuencia:

Pérdida de vidas humanas y fallo en la defensa contra misiles enemigos.

Por la Verdad, por la fe y la ciencia

Pruebas de Software



El Error de Therac-25 (1985-1987)

La máquina de radioterapia Therac-25 administró dosis letales de radiación a varios pacientes.

Motivo:

Fallos de software en el sistema de control de Therac-25. La falta de pruebas rigurosas y la dependencia excesiva del software sin sistemas de seguridad adecuados llevaron a errores que causaron una sobreexposición de radiación.

Consecuencia:

Múltiples muertes y lesiones graves a pacientes, además de daños a la reputación de la empresa y una mayor regulación de las pruebas de software en dispositivos médicos.

Pruebas de Software



El Accidente del Mars Climate Orbiter (1999)

La sonda Mars Climate Orbiter de la NASA se perdió al entrar en la atmósfera marciana y se quemó.

Motivo:

Un error de software debido a la falta de conversión de unidades. El software de navegación del orbitador utilizó unidades inglesas (libras-fuerza) en lugar de unidades métricas (newtons), causando un error de trayectoria.

Consecuencia:

Pérdida de una misión espacial de 125 millones de dólares y valiosos datos científicos.

Por la Verdad, por la fe y la ciencia

Pruebas de Software



El Escándalo del Software de Emisiones de Volkswagen (2015)

Volkswagen fue sorprendido utilizando un software en sus vehículos diésel para manipular las pruebas de emisiones.

Motivo:

El software detectaba cuando el coche estaba siendo probado y ajustaba temporalmente los parámetros del motor para cumplir con los estándares de emisiones.

Consecuencia:

Multas por miles de millones de dólares, retirada de vehículos, pérdida de confianza de los consumidores y daño significativo a la reputación de Volkswagen.

Pruebas de Software

Fallo del Sistema Bancario de RBS (2012)

Un error en la actualización del software de Royal Bank of Scotland (RBS) provocó la caída de sus sistemas bancarios, afectando a millones de clientes

Motivo:

Un fallo en la actualización de software que no se había probado adecuadamente en un entorno controlado. Esto provocó un bloqueo en los sistemas que procesaban pagos y transacciones bancarias.

Consecuencia:

Durante varios días, los clientes no pudieron acceder a sus cuentas ni realizar transacciones. RBS tuvo que pagar multas y compensaciones significativas.



Pruebas de Software

Beizer (BEI90,pag. 1) lo plantea de mejor manera diciendo:

existe un mito que dice que, si fuéramos buenos programando, no habría errores que buscar.

Elemento crítico para la garantía de la calidad del software y representa una revisión final de las especificaciones del diseño y codificación .

A la Verdad, por la fe y la ciencia

Pruebas de Software

Los objetivos de las pruebas

Es un proceso de ejecución de un programa con la intención de descubrir un error.



Casos de
Prueba



Un buen caso de prueba es aquel que tiene una alta probabilidad de mostrar un error no descubierto hasta entonces.

Una prueba tiene éxito si descubre un error no detectado hasta entonces.

A la Verdad, por la fe y la ciencia

Principios de la prueba de software

Las pruebas deberían planificarse mucho antes de que empiecen el desarrollo

El principio de PARETO es aplicable a la prueba del software

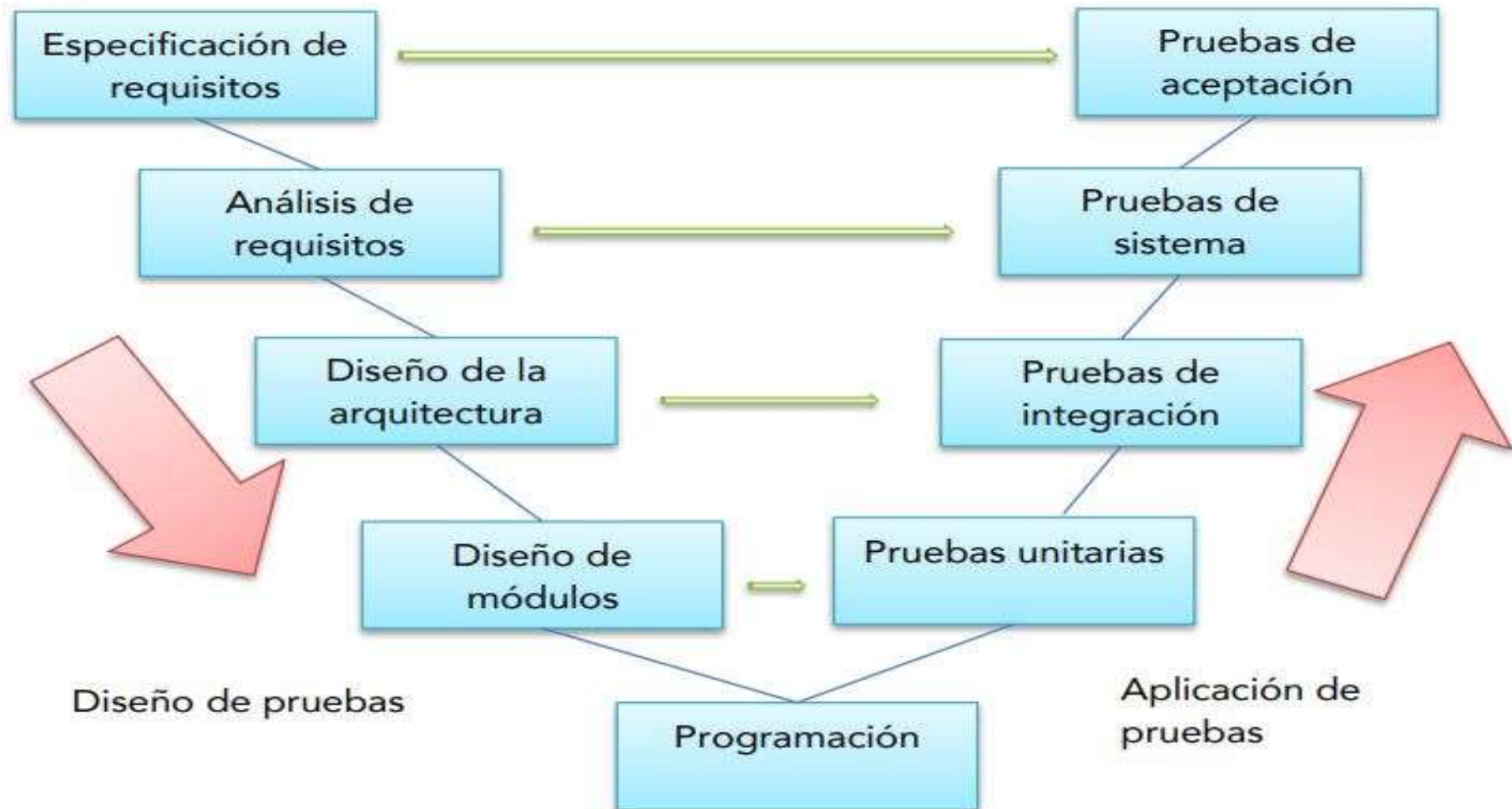
Las pruebas deberían empezar por << lo pequeño>> y progresar hacia << lo grande>>

No son posibles las pruebas exhaustivas.

Para ser más efectivas, las pruebas deberían ser conducidas por un equipo independiente.

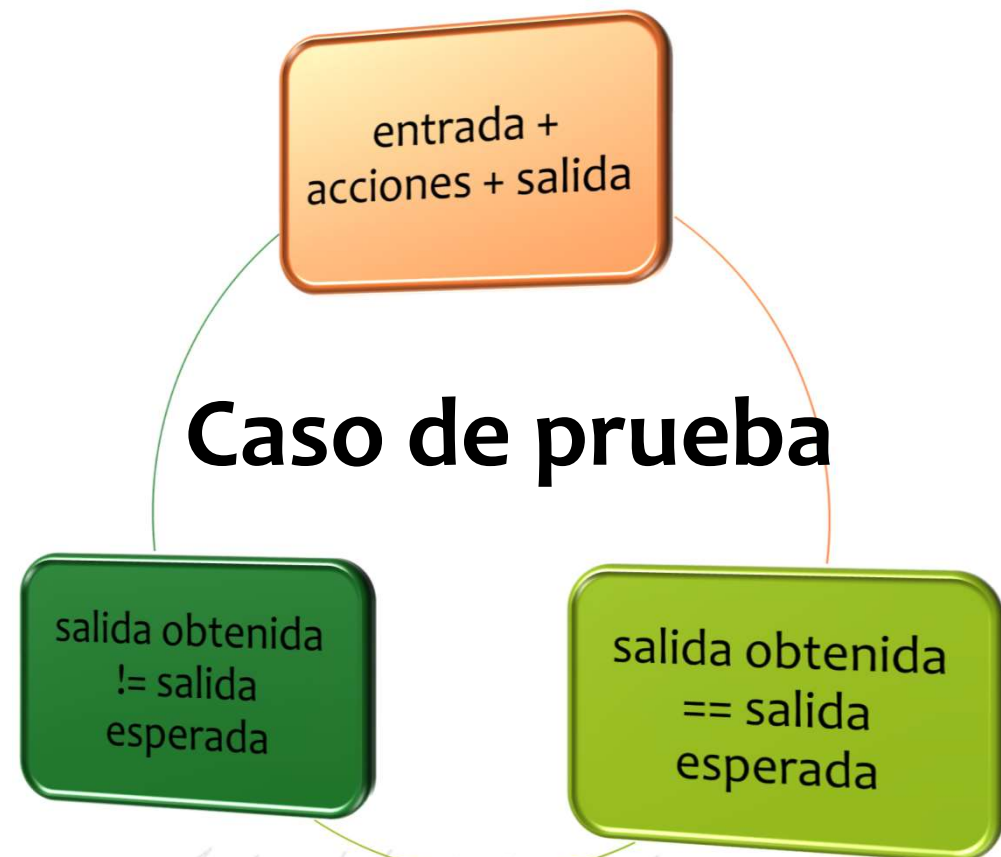
y la ciencia

Ciclo en V



En definitiva

- ☐ Probar es ejecutar casos de prueba.
- ☐ ¿Un programa que pasa todos sus casos de prueba es un programa sin errores?.



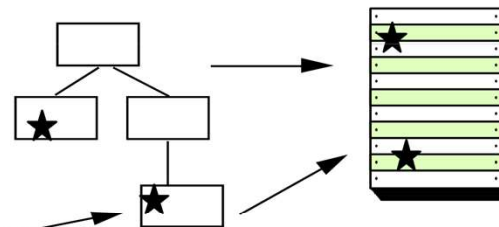
A la Verdad, por la fe y la ciencia

Relación entre Error, Defecto y Fallo

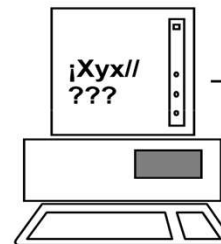
Error
Equivocación
del programador



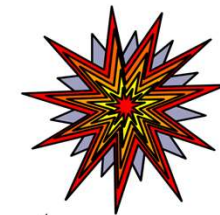
Sistema de gestión de aeropuerto



Defecto (calidad)



S. Aproximación

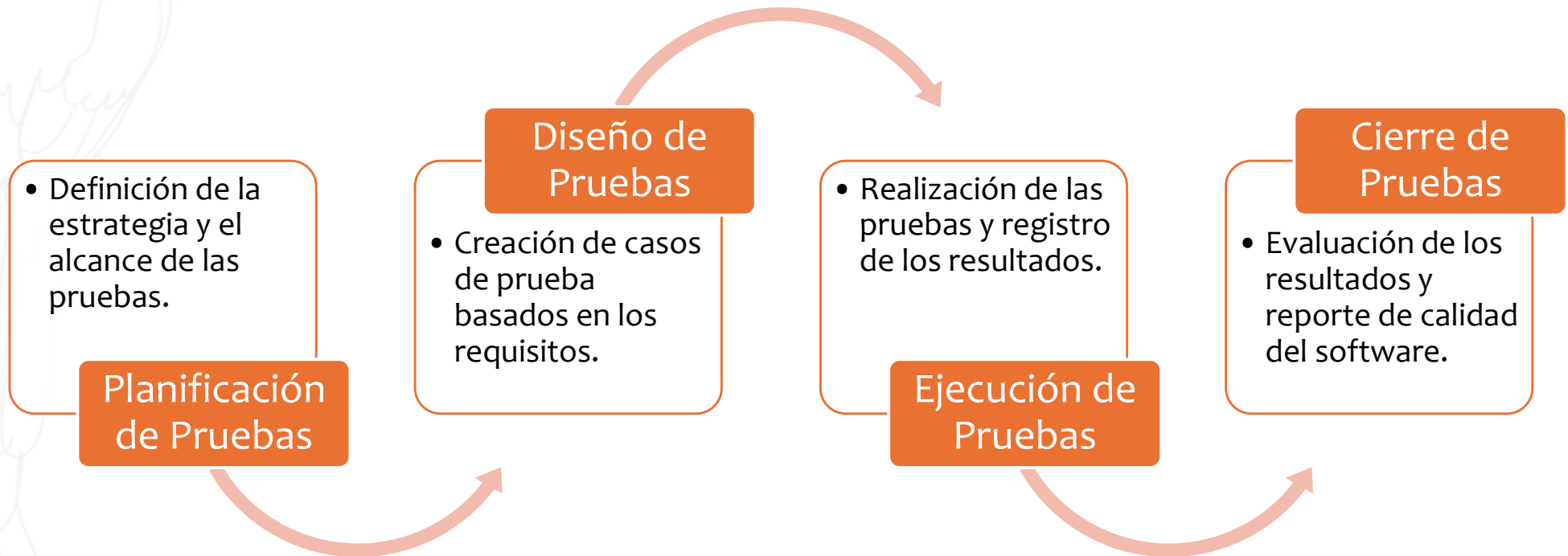


Accidente
(seguridad)

<https://www.youtube.com/watch?v=aCxJ4Qo3bRg>

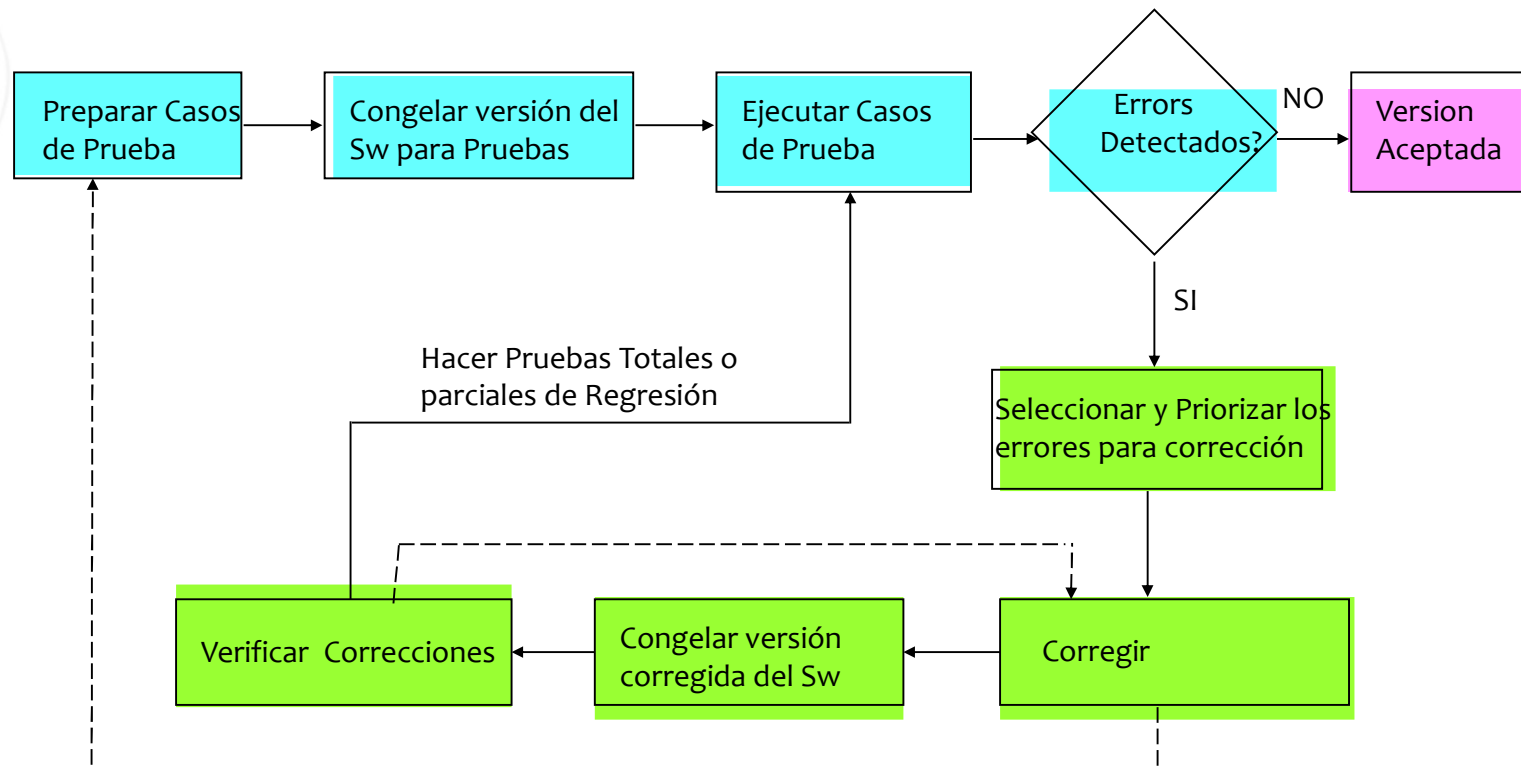
Por la Verdad, por la fe y la ciencia

Ciclo de Vida de las Pruebas de Software



A la Verdad, por la fe y la ciencia

Proceso de Pruebas de Programas



Tipos de Pruebas de Software

Pruebas Funcionales

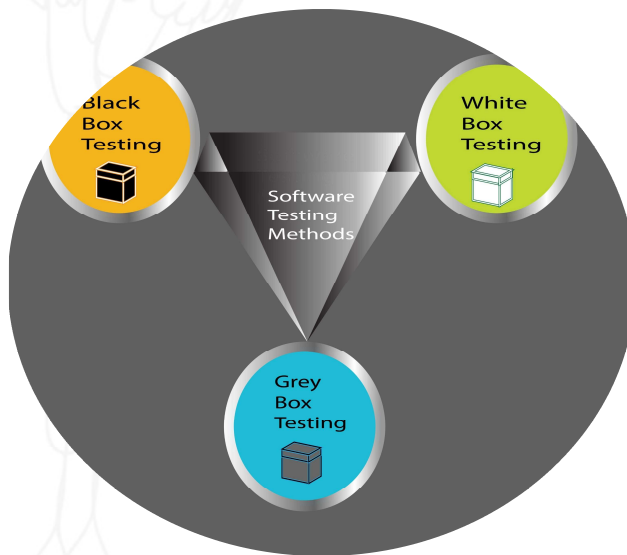
Validan que el software cumple con los requisitos funcionales.

Pruebas No Funcionales:

Evalúan aspectos de calidad como rendimiento, usabilidad, y seguridad.

A la Verdad, por la fe y la ciencia

Según el Conocimiento del Código



Caja Negra (Black Box):

- Se prueba la funcionalidad sin conocer el código interno. El tester se enfoca en las entradas y salidas, como un usuario final.

Caja Blanca (White Box):

- Se prueba la estructura interna, el código y la lógica. Requiere habilidades de programación.

Caja Gris (Grey Box):

- Un híbrido. Se tiene acceso limitado a la estructura interna (como bases de datos o documentación técnica) para diseñar mejores pruebas de usuario.

A la Verdad, por la fe y la ciencia

Los Niveles de Prueba (La Pirámide)



Pruebas Unitarias (Unit Testing): Verifican que un componente individual (una función o clase) funcione correctamente. Son rápidas y automatizadas.



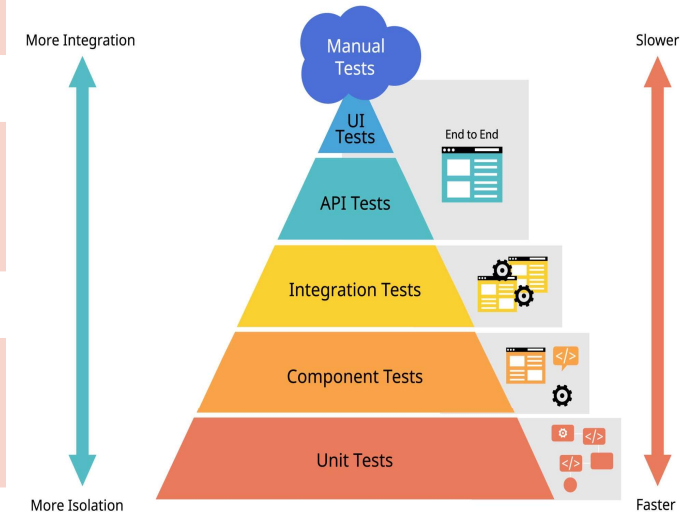
Pruebas de Integración: Aseguran que los diferentes módulos o servicios funcionen bien cuando se conectan entre sí.



Pruebas de Sistema: Se prueba el software completo e integrado para verificar que cumple con los requisitos.



Pruebas de Aceptación: La última palabra la tiene el cliente o el usuario final. ¿El producto resuelve el problema de negocio?



A la Verdad, por la fe y la ciencia

Pruebas de software

Los Niveles de Prueba (La Pirámide) - Aceptación

Pruebas Alfa

- ☐ Prueba de aceptación para detectar errores en el sistema bajo un ambiente controlado.
- ☐ La verificación involucra la ejecución de partes o todo del sistema en ambientes simulados, con el fin de encontrar errores.
- ☐ Se llevan a cabo en el lugar en donde fue desarrollado el sistema

Pruebas Beta

- ☐ Realizar la validación del sistema por parte del usuario.
- ☐ Prueba de aceptación donde La validación (o pruebas beta) involucra el uso del software en un ambiente real.
- ☐ Se selecciona un grupo de usuarios que ponen a trabajar el sistema en un ambiente real. Usan el sistema en sus actividades cotidianas, procesan transacciones y producen salidas normales del sistema

A la Verdad, por la fe y la ciencia

Funcionales vs. No Funcionales

Tipo	¿Qué evalúa?	Ejemplos
Funcionales	El "Qué" hace el sistema.	Pruebas de humo, regresión, pruebas de API.
No Funcionales	El "Cómo" lo hace.	Rendimiento, seguridad, usabilidad, carga.

Pruebas Funcionales: Se centran en los requisitos del negocio.

- ☐ Pruebas de Humo (Smoke Testing): Pruebas rápidas para ver si las funciones básicas sirven (si la app "no echa humo" al encenderla).
- ☐ Pruebas de Regresión: Aseguran que un cambio nuevo no haya roto algo que ya funcionaba.

Pruebas No Funcionales: Se centran en la experiencia y calidad técnica.

- ☐ Pruebas de Rendimiento (Performance): Evalúan velocidad y estabilidad.
- ☐ Pruebas de Usabilidad: Evalúan qué tan intuitivo es el software.

Referencias:

- ❑ Kaner, C., Bach, J., & Pettichord, B. (2002). Lessons Learned in Software Testing. Wiley. ISBN: 978-0471081128.
- ❑ Myers, G. J., Sandler, C., & Badgett, T. (2011). The Art of Software Testing (3rd ed.). Wiley. ISBN: 978-1118031964.
- ❑ Jorgensen, P. C. (2013). Software Testing: A Craftsman's Approach (4th ed.). Auerbach Publications. ISBN: 978-1466560680.
- ❑ Crispin, L., & Gregory, J. (2009). Agile Testing: A Practical Guide for Testers and Agile Teams. Addison-Wesley. ISBN: 978-0321534460.
- ❑ Graham, D., Black, R., & van Veenendaal, E. (2012). Foundations of Software Testing: ISTQB Certification (3rd ed.). Cengage Learning. ISBN: 978-1408044056.

A la Verdad, por la fe y la ciencia

Referencias:

- ❑ Whittaker, J. A. (2009). Exploratory Software Testing. Addison-Wesley. ISBN: 978-0321636416.
- ❑ Kinsbruner, E., Craske, A., & Feloney, S. (2019). Continuous Testing for DevOps Professionals. Apress. ISBN: 978-1484253505.
- ❑ Weinberg, G. M. (2008). Perfect Software and Other Illusions About Testing. Dorset House. ISBN: 978-0932633699.
- ❑ Whittaker, J. A. (2002). How to Break Software: A Practical Guide to Testing. Addison-Wesley. ISBN: 978-0201796193.
- ❑ Black, R. (2012). Advanced Software Testing - Vol. 1: Guide to the ISTQB Advanced Certification as an Advanced Test Analyst (2nd ed.). Rocky Nook. ISBN: 978-1937538497.

A la Verdad, por la fe y la ciencia



¡Gracias!